FastAPI na Prática: Criando APIs Modernas em Python

Asimov Academy

ASIMOV

Conteúdo

1. Introdução ao FastAPI – Construindo APIs de Forma Profissional **4**
O que vamos construir? . . . 4
Por que entender os Métodos HTTP? . . . 4
O que você vai aprender? . . . 4

2. A Arquitetura do FastAPI – Servidores, Protocolos e Assincronismo **6**
O FastAPI não trabalha sozinho . . . 6
O Papel do Uvicorn e o Protocolo ASGI . . . 6
O Fluxo de uma Requisição: Do Cliente ao Código . . . 6
Por que a Programação Assíncrona é Vital? . . . 7

3. Modelagem de Dados e a Implementação do Método GET **9**
Definição dos Dados e Identificação Única (UUID) . . . 9
Preparando o Ambiente de Desenvolvimento . . . 9
Criando a Base da Aplicação(`api.py`) . . . 10
Validação com Pydantic . . . 11
Implementando o Endpoint de Leitura (GET) . . . 11
Executando a API e Explorando o Swagger . . . 13

4. Consumo de APIs e a Busca por Identificador Único **14**
Preparando o Ambiente do Cliente . . . 14
Estruturando o Script do Cliente (`cliente.py`) . . . 14
Implementando a Busca por ID (Lado do Servidor) . . . 15
Conectando o Cliente ao Endpoint de Busca . . . 16
Por que isso é importante? . . . 16

5. O Método POST – Criando Novos Registros **18**
O conceito do Método POST . . . 18
Implementação no Servidor (`api.py`) . . . 19
Testando a Integração . . . 20

6. O Método PUT – Substituição Total de Registros **22**
O Conceito do Método PUT . . . 22
Implementação no Servidor (`api.py`) . . . 23
Implementação no Cliente (`cliente.py`) . . . 23
Atualizando o Menu do Cliente . . . 24

7. O Método PATCH – Realizando Atualizações Parciais **25**
PATCH vs. PUT: Qual a diferença? 25
Modelagem de Dados com Campos Opcionais 25
Implementação no Servidor (`api.py`) 26
Implementação no Cliente (`cliente.py`) 26
Atualizando o Menu e Testando 27

8. O Método DELETE – Removendo Registros com Segurança **28**
O Modelo de Resposta para Confirmação 28
Implementação no Servidor (`api.py`) 28
Implementação no Cliente (`cliente.py`) 29
Atualização Final do Menu 29
Testando o Fluxo de Exclusão 29
Conclusão do Módulo Base 30

9.Persistência e Ferramentas – Instalando o DBeaver **31**
Por que utilizar o DBeaver? 31
Instalando o DBeaver 31
Instalação no Windows 31
Instalação no Linux (Base Debian/Ubuntu) 32
Primeira Conexão com SQLite 32

10. Persistência de Dados com SQLModel e Refatoração do Projeto **34**
O Papel do SQLModel e o Conceito de ORM 34
Reestruturação do Projeto 34
Modelagem Avançada em `models.py` 35

11. Persistência Real – Criando a Base de Dados e o Ciclo de Vida da API **37**
Configurando o Motor do Banco de Dados (`database.py`) 37
Integrando ao FastAPI com o Ciclo de Vida (`main.py`) 38
Verificando a Estrutura com DBeaver 38

12. Refatoração e Organização de Rotas com APIRouter **40**
Modularização com APIRouter 40
Injeção de Dependência de Sessão 40
Implementando a Busca com SQLModel 41
Registro das Rotas no Arquivo Principal 41
Dados Iniciais para Testes (Default Data) 41

13. Consultas Individuais e Criação de Dados com SQLModel **44**
Recuperando um Registro Específico (GET por UUID) 44
O Método POST com Persistência Real 45
Validando a Persistência no Banco de Dados 45

14. Finalizando o CRUD – PUT, PATCH e DELETE com SQLModel **47**
O Método PUT: Substituição Integral 47
O Método PATCH: Atualização Parcial e Filtros 47
O Método DELETE: Remoção Permanente 48
A Transparência da Refatoração 49

15. Paginação e Importação em Lote **50**
A Necessidade da Paginação 50
Preparação e Limpeza do Ambiente 50
Importação de Livros via CSV 50
Executando a Importação em Lote 51

16. Implementando a Paginação no FastAPI **53**
Modificando o Endpoint de Listagem 53
Lógica de Cálculo e Consultas ao Banco 53
Implementação Completa do Código 54
Testando na Documentação (Swagger) 54

17. Paginação no Cliente Python **56**
Lógica de Acúmulo de Dados 56
Obtendo a Primeira Página e Metadados 56
Percorrendo as Demais Páginas 57
Exibição do Resultado Final 57
Conclusão do projeto 57

18. Síntese e Consolidação do Aprendizado **59**
1. A Arquitetura: FastAPI, Uvicorn e ASGI 59
2. Do In-Memory à Persistência com SQLModel 59
3. O Ciclo de Vida CRUD e a Validação com Pydantic 59
4. Organização Modular e Escalabilidade 60
5. Lidando com Grandes Volumes: Paginação 60
Conclusão 60

1. Introdução ao FastAPI – Construindo APIs de Forma Profissional

Bem-vindo ao início da sua jornada com o **FastAPI**. Este material tem como objetivo situar você no universo das APIs, apresentando os conceitos fundamentais que guiarão todo o nosso aprendizado, com exemplos consistentes e de fácil entendimento.

O que vamos construir?

O foco será aprender a criar uma API para o gerenciamento de livros. O objetivo não é apenas copiar código, mas entender a lógica por trás de cada funcionalidade e como a comunicação **HTTP** realmente funciona.

Começaremos de forma simples, utilizando uma **base de dados em memória** (como listas ou dicionários no Python) para que você possa focar no que realmente importa no início: a lógica da API e a comunicação entre cliente e servidor. Conforme avançarmos, evoluiremos para uma estrutura profissional, utilizando identificadores únicos (**UUID**), modelos de dados bem definidos e integração com bancos de dados reais.

Por que entender os Métodos HTTP?

Um dos pilares do desenvolvimento de back-end é saber como o cliente se comunica com o servidor. Ao longo das lições, você verá na prática o que acontece quando uma aplicação faz requisições utilizando os principais métodos HTTP:

- **GET:** Para buscar informações.
- **POST:** Para criar novos dados.
- **PUT e PATCH:** Para atualizar informações existentes.
- **DELETE:** Para remover dados.

Além de construir o servidor com FastAPI, você também verá como construir e adaptar um **cliente Python** para consumir esses dados, garantindo uma visão completa do ecossistema de uma integração.

O que você vai aprender?

Ao concluir este material, você terá a segurança necessária para trabalhar com back-end, automações e sistemas reais. Os principais objetivos são:

1. **Compreender o funcionamento real de uma API:** Entender a relação entre o FastAPI e o servidor **Uvicorn**.
2. **Dominar os métodos HTTP:** Saber exatamente quando e por que usar GET, POST, PUT, PATCH e DELETE.
3. **Organização de Código:** Criar APIs de forma clara, organizada e pronta para evoluir para ambientes de produção.
4. **Consumo de APIs:** Desenvolver clientes em Python capazes de interagir com qualquer API.

Entender o FastAPI é entender o coração de muitas tecnologias modernas de back-end. Prepare seu ambiente, pois no próximo capítulo mergulharemos na arquitetura técnica e nas ferramentas que fazem o FastAPI ser tão rápido e eficiente.

Bora criar a nossa primeira API!

2. A Arquitetura do FastAPI – Servidores, Protocolos e Assincronismo

Para desenvolver aplicações eficientes, não basta apenas escrever o código da lógica de negócio; é fundamental compreender como a sua aplicação se comunica com o mundo exterior. Neste capítulo, exploraremos a infraestrutura que sustenta o **FastAPI**, desmistificando o papel do servidor e os protocolos que garantem a alta performance dessa ferramenta.

O FastAPI não trabalha sozinho

Embora frequentemente nos referimos ao FastAPI como o "servidor", tecnicamente ele é uma **aplicação ASGI** (*Asynchronous Server Gateway Interface*). Na prática, o FastAPI não possui a capacidade nativa de compreender ou processar o protocolo **HTTP** de forma isolada; ele não sabe "servir" uma página ou dados para a web por conta própria.

Para que a comunicação ocorra, o FastAPI precisa de dois componentes essenciais:

1. **Um Servidor Web:** No caso do FastAPI, o padrão utilizado é o **Uvicorn**.
2. **Um Protocolo de Interface:** O **ASGI**, que funciona como a "ponte" ou tradutor entre o servidor e a aplicação.

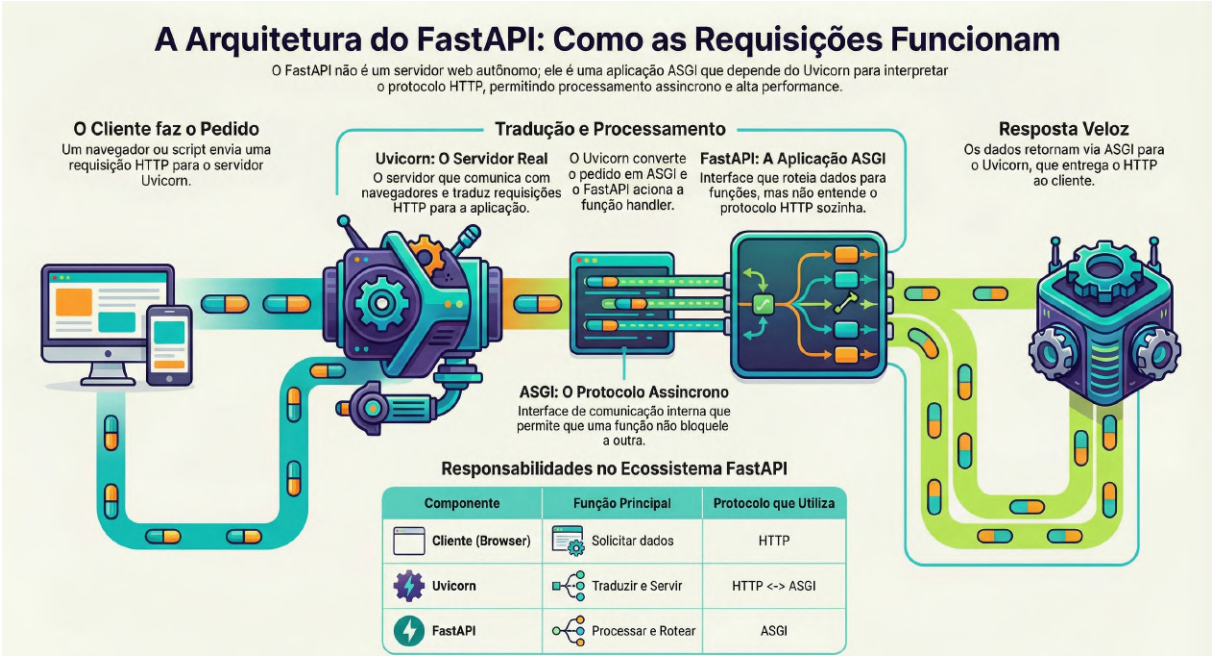
O Papel do Uvicorn e o Protocolo ASGI

O **Uvicorn** é o componente que realmente "conversa" com o navegador, com aplicativos ou com scripts que solicitam dados. Ele recebe as requisições via HTTP e as traduz para o protocolo **ASGI**, que é uma interface específica do Python para comunicação assíncrona.

O **ASGI** é o sucessor do antigo padrão WSGI. O grande diferencial está no "A" de **Asynchronous** (Assíncrono). Esse protocolo permite que a comunicação entre o servidor (Uvicorn) e a aplicação (FastAPI) ocorra sem bloqueios, o que é um dos principais motivos para a velocidade excepcional do FastAPI.

O Fluxo de uma Requisição: Do Cliente ao Código

Para entender como esses componentes interagem, vamos observar o caminho que os dados percorrem quando um cliente solicita, por exemplo, uma lista de livros:



1. **A Requisição do Cliente:** Um cliente (navegador ou script) faz uma requisição **HTTP GET** para um endereço (ex: `biblioteca.com/livros`).
2. **A Tradução do Servidor:** O **Uvicorn** recebe essa requisição e a traduz para o protocolo **ASGI**, enviando-a para o FastAPI.
3. **O Roteamento do FastAPI:** O FastAPI recebe o sinal e identifica qual função no seu código deve responder àquele pedido (o "handler").
4. **A Execução Assíncrona:** A função (geralmente definida como `async def`) busca os dados — seja em memória ou em um banco de dados — e os retorna.
5. **A Resposta Final:** O FastAPI devolve os dados via ASGI para o Uvicorn, que os traduz de volta para uma resposta **HTTP** compreensível para o cliente.

Por que a Programação Assíncrona é Vital?

O uso de funções **assíncronas** permite que o servidor processe múltiplas chamadas simultaneamente. Em servidores tradicionais (síncronos), uma requisição lenta poderia "travar" o processamento de outras. No FastAPI, enquanto uma função aguarda uma resposta do banco de dados, o servidor fica livre para atender outras solicitações, garantindo que o sistema seja extremamente ágil e capaz de lidar com grandes volumes de tráfego.

Ao iniciar sua aplicação com comandos como `fastapi dev`, o Uvicorn é iniciado internamente para possibilitar essa arquitetura. Compreender essa separação entre **servidor (Uvicorn)** e **apli-**

cação (FastAPI) é o que diferencia um desenvolvedor iniciante de um profissional preparado para criar sistemas escaláveis.

No próximo capítulo, começaremos a aplicar esses conceitos na prática, criando nossos primeiros *endpoints* e estruturando nossa base de dados inicial.

3. Modelagem de Dados e a Implementação do Método GET

Neste capítulo, sairemos da teoria arquitetural e entraremos na fase prática do desenvolvimento. Você aprenderá a estruturar os dados da sua API, garantindo que as informações sejam validadas e transmitidas de forma segura utilizando o método **GET**.

Definição dos Dados e Identificação Única (UUID)

Uma API lida essencialmente com informações organizadas. Para o nosso projeto, cada livro terá os seguintes campos: **Autor, Título, Editora e Ano**. No entanto, para que o sistema consiga gerenciar esses dados de forma eficiente, precisamos de um identificador.

Existem duas formas principais de identificar registros:

• **IDs Inteiros:** Comuns em bancos de dados SQL tradicionais, onde cada linha recebe um número sequencial (1, 2, 3…). O problema é que eles prendem o dado a uma tabela específica.

• **UUID (Universally Unique Identifier):** É um identificador universal que não depende da posição do dado na tabela. O uso de UUID é uma **boa prática em APIs e microsserviços**, pois garante a consistência dos dados mesmo em sistemas distribuídos ou durante migrações de banco de dados.

ID	uuid	autor	titulo	editora	ano
1	4477052b-c873-4d89-a4f0-1cf4f027ecbc	George Orwell	1984	Companhia das Letras	1949
2	d123f416-2c6f-4e72-9e61-cbfc472db90b	J.R.R. Tolkien	O Hobbit	HarperCollins	1937

Preparando o Ambiente de Desenvolvimento

Para garantir que seu projeto seja organizado e não interfira em outras bibliotecas do seu computador, utilizaremos um **Ambiente Virtual (VEnv)**.

No terminal, execute os comandos abaixo para configurar seu ambiente e instalar o FastAPI:

```
# Criando o ambiente virtual
python -m venv venv

# Ativando o ambiente (Linux/Mac)
source venv/bin/activate

# Ativando o ambiente (Windows)
```

```
.\venv\Scripts\activate

# Instalando o FastAPI
pip install "fastapi[standard]"
```

Para uma melhor experiência no VS Code, sugerimos extensões como **Black Formatter** (formatação), **Pylance** (verificação de tipos) e **Python Debugger**.

Criando a Base da Aplicação(`api.py`)

Agora que nosso ambiente está configurado, vamos começar criando e estruturando o arquivo`app.py`. O primeiro passo no código é instanciar o FastAPI:

```
from fastapi import FastAPI

# 1. Inicialização da API
app = FastAPI(title="API de Livros")
```

Nesta fase inicial, utilizaremos um **dicionário Python em memória** para simular nosso banco de dados. Isso permite focar na lógica da API antes de partirmos para bancos de dados complexos. Para isso iremos importar a biblioteca UUID para definirmos o tipo do ID como `uuid4()`, isso fará com que o código gere automaticamente um UUID para cada livro.

```
from fastapi import FastAPI
from uuid import uuid4


# 1. Inicialização da API
app = FastAPI(title="API de Livros")

# 2. Base de Dados em Memória (Dicionário)
livros_db = {
    1: {
    "id": uuid4(),
    "autor": "George Orwell",
    "titulo": "1984",
    "editora": "Companhia das Letras",
    "ano": 1949
    },
    2: {
    "id": uuid4(),
    "autor": "J.R.R. Tolkien",
    "titulo": "O Hobbit",
    "editora": "HarperCollins",
    "ano": 1937
    }
}
```

Validação com Pydantic

Para garantir que a API receba e envie dados no formato correto, utilizamos a biblioteca **Pydantic**. Através da classe `BaseModel`, criamos um "molde" (modelo) para o livro.

Por que usar o Pydantic? O Pydantic valida automaticamente os tipos de dados. Se a API espera um número no campo "ano" e recebe um texto, ele gera um erro automaticamente, garantindo a segurança e consistência da aplicação.

Criaremos um modelo chamado `Livro` que herda de `BaseModel`:

```
from fastapi import FastAPI
from uuid import UUID, uuid4
from pydantic import BaseModel


# 1. Inicialização da API
app = FastAPI(title="API de Livros")

# 2. Base de Dados em Memória (Dicionário)
livros_db = {
    1: {
    "id": uuid4(),
    "autor": "George Orwell",
    "titulo": "1984",
    "editora": "Companhia das Letras",
    "ano": 1949
    },
    2: {
    "id": uuid4(),
    "autor": "J.R.R. Tolkien",
    "titulo": "O Hobbit",
    "editora": "HarperCollins",
    "ano": 1937
    }
}

# 3. Definição do Modelo de Dados
class Livro(BaseModel):
    id: UUID
    autor: str
    titulo: str
    editora: str
    ano: int
```

Implementando o Endpoint de Leitura (GET)

O método **GET** é utilizado exclusivamente para a extração de dados. Implementaremos uma função assíncrona para listar todos os livros da nossa base em memória.

Antes de criar o endpoint, vamos adicionar mais uma importação:

```
from typing import List
```

A biblioteca typing serve para fornecer **“type hints”** no código. Esses **“type hints”** são uma sintaxe especial que permite declarar o tipo de uma variável.

Adicione o seguinte código ao seu arquivo `api.py`:

```
# GET - Listar todos os livros
@app.get(path="/livros", response_model=List[Livro])
async def listar_livros() -> List[Livro]:
    # Retorna uma lista de objetos Livro a partir dos valores do dicionário
    return [Livro(**dados) for dados in livros_db.values()]
```

Neste trecho, o decorador `@app.get("/livros")` define a rota que o cliente deve acessar. O parâmetro `response_model` instrui o FastAPI a converter automaticamente nossos dados para o formato JSON correto, seguindo o modelo que definimos em `class Livro(BaseModel)`.

Uso do operador `**` em Python Você pode estar se perguntando o porquê de usarmos o operador `**`. Em muitos casos, ele é usado em operações matemáticas para representar a potência. Porém, quando utilizado com dicionários, esse operador serve para desempacotar os dados, ou seja, ele pega cada par `chave: valor` e os passa como parâmetros nomeados. Assim, conseguimos criar um objeto usando diretamente os valores do dicionário, desde que as chaves tenham o mesmo nome dos atributos da classe.

Exemplo prático Considere o seguinte dicionário com informações de um livro:

```
dados = {
    "titulo": "Dom Casmurro",
    "autor": "Machado de Assis",
    "ano": 1899
}
```

E uma classe `Livro` definida da seguinte forma:

```
class Livro:
    def __init__(self, titulo, autor, ano):
        self.titulo = titulo
        self.autor = autor
        self.ano = ano
```

Ao utilizar o operador `**`, podemos criar um objeto `Livro` da seguinte maneira:

```
livro = Livro(**dados)
```

Esse código é equivalente a escrever:

```
livro = Livro(
    titulo="Dom Casmurro",
    autor="Machado de Assis",
    ano=1899
)
```

Observação importante Para que o uso do operador `**` funcione corretamente, as **chaves do dicionário devem ter o mesmo nome** dos parâmetros definidos no construtor da classe. Caso contrário, ocorrerá um erro.

Executando a API e Explorando o Swagger

Para colocar sua API no ar, utilize o comando abaixo no terminal:

```
fastapi dev api.py
```

O servidor será iniciado em `http://127.0.0.1:8000`. O FastAPI gera automaticamente uma documentação interativa no endereço `/docs` (conhecida como **Swagger UI**).

Ao acessar a documentação, você poderá:

• Visualizar o método **GET /livros**.

• Testar a requisição clicando em **"Try it out"** e depois em **"Execute"**.

• Verificar o código de status **200 OK**, que indica sucesso, e receber os dados no formato **JSON**.

Caso ocorra algum erro de digitação na URL, a API retornará o status **404 Not Found**, informando que o recurso não foi localizado.

Neste capítulo, você construiu a base sólida de uma API: modelou os dados, configurou o ambiente e implementou a leitura de informações. No próximo capítulo, aprenderemos a consumir nossa API através de um cliente externo e implementar a busca por um livro através do UUID.

4. Consumo de APIs e a Busca por Identificador Único

No capítulo anterior, estabelecemos a base da nossa API e aprendemos a listar todos os registros. Agora, avançaremos para duas competências fundamentais: a criação de um **cliente externo** em Python para consumir esses dados e a implementação da lógica para localizar um **único registro** através do seu identificador único (UUID).

Preparando o Ambiente do Cliente

Para que um script Python consiga se comunicar com a nossa API, precisamos de uma biblioteca capaz de realizar requisições HTTP. A biblioteca padrão para essa tarefa é a `requests`.

No seu terminal, com o ambiente virtual (VEnv) ativado, realize a instalação:

```
pip install requests
```

Estruturando o Script do Cliente (`cliente.py`)

O cliente será um arquivo independente chamado `cliente.py`. O primeiro passo é definir a localização da nossa API através de uma constante global e criar a estrutura básica de interação.

A função `menu` utiliza um laço de repetição `while True` para que o programa continue rodando até que o usuário decida sair. Para evitar de sempre estarmos escrevendo a URL da API ao tentar fazer uma requisição, criamos uma variável global para armazenar essa URL e chamar apenas a variável.

```
import requests
import json

# URL base da nossa API
API_URL = "http://127.0.0.1:8000"

def menu():
    while True:
        print("\n" + "="*30)
        print("CLIENTE API DE LIVROS")
        print("="*30)
        print("1. Listar livros")
        print("2. Obter livro por UUID")
        print("0. Sair ")

        # .strip() remove espaços extras
        opcao = input("\nEscolha uma opção: ").strip()

        if opcao == "1":
            listar_livros()
        elif opcao == "2":
```

```
            obter_livro_por_id()
        elif opcao == "0":
            print("Encerrando cliente...")
            break
        else:
            print("Opção inválida!")

if __name__ == "__main__":
    menu() # Inicia o programa
```

Com o nosso menu criado, vamos agora implementar as funções `listar_livros()` e `obter_livro_por_id()`. Mas antes de criarmos essas funções precisamos implementar uma função que vai fazer o tratamento das respostas que o cliente vai entregar para o usuário e adicionar na nossa API o endpoint de busca de livros através do UUID. ## Tratamento Universal de Respostas

Para evitar repetição de código, criaremos uma função auxiliar chamada `tratar_resposta`. Ela será responsável por verificar se a comunicação com o servidor foi bem-sucedida (Status 200) ou se houve algum erro (Status 400 ou superior), além de formatar o JSON de forma legível (indentado).

```
def tratar_resposta(resp):
    try:
        # Tenta converter a resposta para JSON
        data = resp.json()
    except ValueError:
        print(f"Status: {resp.status_code}")
        print("Resposta sem JSON")
        print(resp.text)
        return

    # Se o status for de erro (>= 400), exibe a mensagem de erro formatada
    if resp.status_code >= 400:
        print(f"Erro HTTP: {resp.status_code}")

    # Exibe o JSON formatado com 4 espaços de indentação
    print(json.dumps(data, indent=4, ensure_ascii=False))
```

Implementando a Busca por ID (Lado do Servidor)

Antes de o cliente solicitar um livro específico, o servidor (FastAPI) precisa estar preparado para processar essa busca. No arquivo `api.py`, implementaremos o método **GET** com um parâmetro de rota `{livro_id}`.

Utilizaremos a exceção `HTTPException` para retornar o código **404** caso o livro não seja encontrado na nossa base de dados em memória. Vamos importar no nosso código:

```
from fastapi import FastAPI, HTTPException
```

Agora implementaremos o endpoint com a lógica de busca de um livro específico:

```
@app.get(path="/livros/{livro_id}", response_model=Livro, responses={404: {"description":
↪  "Livro não encontrado"}})
async def obter_livro(livro_id: UUID) -> Livro:
    # Percorre os valores do dicionário em memória
    for livro in livros_db.values():
        if livro["uuid"] == livro_id:
            return Livro(**livro)

    # Se o loop terminar sem retorno, o livro não existe
    raise HTTPException(status_code=404, detail="Livro não encontrado")
```

Explicação Técnica: • **Roteamento:** O parâmetro `{livro_id}` na URL está diretamente ligado ao argumento `livro_id: UUID` da função.

• **Tratamento de Erros:** É essencial lançar uma `HTTPException` com status **404** caso o UUID fornecido não exista. Isso informa ao cliente que o recurso para atualização não foi localizado.

• **Documentação (Swagger):** O parâmetro `responses` no decorador garante que a possibilidade do erro 404 apareça na documentação interativa da API.

Conectando o Cliente ao Endpoint de Busca

Agora, voltamos ao arquivo `cliente.py` para implementar as funções de busca e listagem dos livros. Para cada função, utilizaremos `tratar_resposta`, que vai se encarregar de devolver os resultados no formato JSON.

```
def obter_livro_por_id():
    uuid_livro = input("Digite o UUID do livro: ").strip()
    print("\nDetalhes do livro:")

    # Realiza o GET concatenando o UUID na URL
    resp = requests.get(f"{API_URL}/livros/{uuid_livro}")
    tratar_resp(resp)

def listar_livros():
    print("\nListando todos os livros:")
    resp = requests.get(f"{API_URL}/livros")
    tratar_resp(resp)
```

Por que isso é importante?

Neste fluxo, você aprendeu que:

1. **O Cliente** solicita dados via HTTP para o **Servidor (Uvicorn)**.
2. **O FastAPI** roteia essa solicitação para a função correta (`obter_livro`).

3. **A Validação** ocorre tanto no recebimento do UUID quanto na resposta formatada pelo Pydantic.
4. **A Comunicação** entre os dois scripts é o que chamamos de integração de sistemas, a base do desenvolvimento de back-end moderno.

Dica de Estudo: Execute o comando `fastapi dev api.py` em um terminal e o `python cliente.py` em outro. Liste os livros (Opção 1), copie um UUID da lista e use-o na Opção 2 para ver a busca individual funcionando!

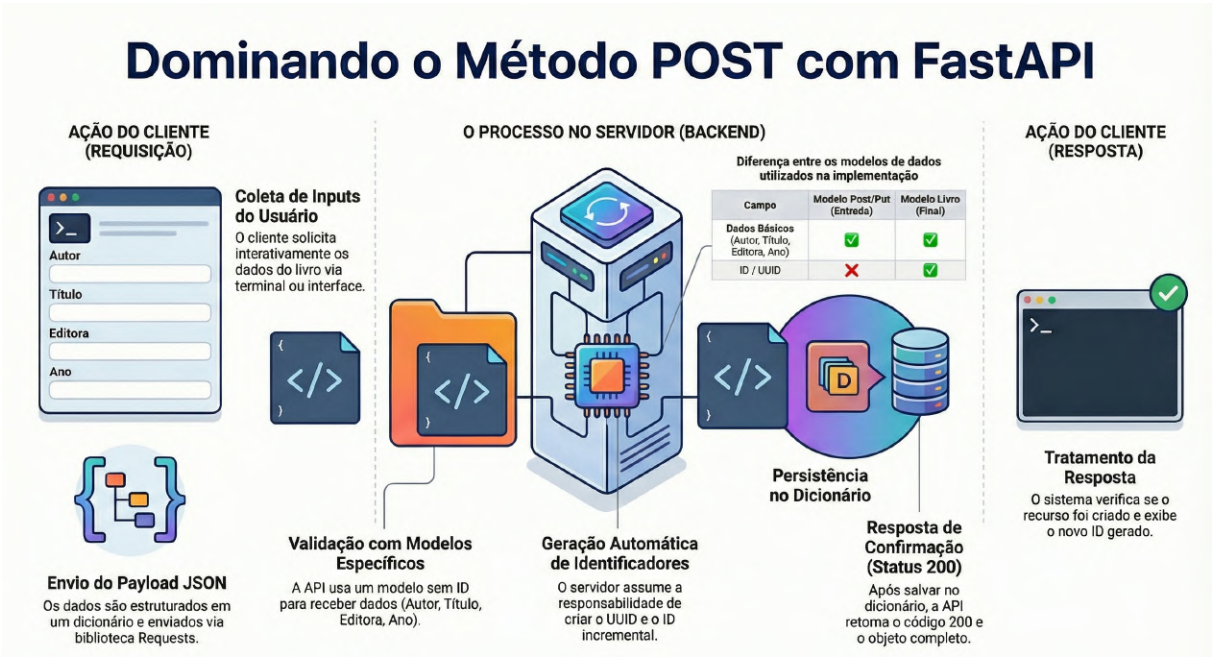
5. O Método POST – Criando Novos Registros

Neste capítulo, aprenderemos a adicionar novos dados à nossa API. Enquanto o método GET é utilizado para leitura, o método **POST** é o padrão para a criação de recursos. Veremos como o servidor processa essas informações, como validar o que o usuário envia e como atualizar o nosso cliente Python para suportar essa nova funcionalidade.

O conceito do Método POST

Diferente do GET, onde os dados de consulta geralmente vão na URL, no método POST as informações são enviadas no **corpo da requisição** (*body*) em formato **JSON**.

O fluxo funciona assim:



1. O cliente envia os dados (Autor, Título, Editora e Ano).
2. A API recebe e valida esses dados.
3. O servidor gera as identificações necessárias (como o UUID) e salva o registro,.
4. A API retorna o objeto criado com um código de status (geralmente **200 OK** ou **201 Created**) para confirmar o sucesso,.

Implementação no Servidor (`api.py`)

No FastAPI, precisamos separar o que o usuário **envia** do que o servidor **armazena**. O cliente não deve enviar o ID ou o UUID; esses campos são de responsabilidade da aplicação.

Criando um Modelo Específico Para isso, no arquivo `api.py`, criamos um modelo de dados que não contém o campo ID, chamado `LivroPostPut`:

```
class LivroPostPut(BaseModel):
    autor: str
    titulo: str
    editora: str
    ano: int
```

Criando o Endpoint de Adição de Livros A função utilizará o decorador `@app.post` e será responsável por gerar o identificador único e salvar o livro na nossa base de dados em memória.

```
@app.post("/livros", response_model=Livro)
async def adicionar_livro(livro: LivroPostPut) -> Livro:
    # 1. Gerar um novo UUID
    novo_uuid = uuid4()

    # 2. Gerar a nova chave para o dicionário (ID incremental)
    novo_id = max(livros_db.keys()) + 1 if livros_db else 1

    # 3. Criar o objeto completo do livro
    livro_gravado = Livro(
        id=novo_uuid,
        autor=livro.autor,
        titulo=livro.titulo,
        editora=livro.editora,
        ano=livro.ano
    )

    # 4. Salvar no "banco de dados" (dicionário) e retorna o novo livro
    livros_db[novo_id] = livro_gravado.model_dump()
    return livro_gravado
```

A função `max()` encontra o maior ID que existe na nossa base de dados e ao fazer isso adiciona + 1, criando um novo ID para o próximo livro que for adicionado. A condição `if livros_db else 1` serve para verificar se o dicionário está vazio: se estiver ele acrescenta o ID 1 ao livro, sendo o primeiro registro, e se não estiver ele executa `max(livros_db.keys()) + 1`.

O `model_dump()` é um método do **Pydantic** usado para transformar o objeto `livro_gravado` em um dicionário. ## Implementação no Cliente (`cliente.py`)

Agora que o servidor está pronto, precisamos atualizar nosso script de cliente para permitir que o usuário digite os dados de um novo livro pelo terminal.

Função de Adição de Livros Criaremos a função `adicionar_livro`, que solicita os dados ao usuário, monta um dicionário (*payload*) e faz a requisição POST.

```
def adicionar_livro():
    print("\nDigite os dados do novo livro:")
    autor = input("Autor: ")
    titulo = input("Título: ")
    editora = input("Editora: ")
    ano = int(input("Ano de publicação: "))

    # Montando o JSON (dicionário Python)
    payload = {
        "autor": autor,
        "titulo": titulo,
        "editora": editora,
        "ano": ano
    }

    # Enviando a requisição POST
    resp = requests.post(f"{API_URL}/livros", json=payload)
    print("\nLivro adicionado!")
    tratar_resp(resp)
```

Atualizando o Menu Para finalizar, adicionamos a nova opção ao menu principal do cliente:

```
def menu():
    while True:
        # ... opções anteriores
        print("3. Adicionar novo livro")
        print("0. Sair")

        opcao = input("\nEscolha uma opção: ").strip()

        if opcao == "1":
            listar_livros()
        elif opcao == "2":
            obter_livro_por_id()
        elif opcao == "3":
            adicionar_livro()
        # ... restante do código
```

Testando a Integração

Com o servidor rodando (`fastapi dev api.py`), você pode testar a criação de duas formas:

1. **Pelo Swagger UI:** Acesse `/docs`, abra o método POST, clique em “Try it out”, preencha o JSON e execute.
2. **Pelo Cliente Python:** Execute `python cliente.py`, escolha a opção 3 e preencha os dados solicitados.

Se tudo estiver correto, ao listar os livros novamente (opção 1), o novo registro aparecerá com seu UUID gerado automaticamente pelo servidor,.

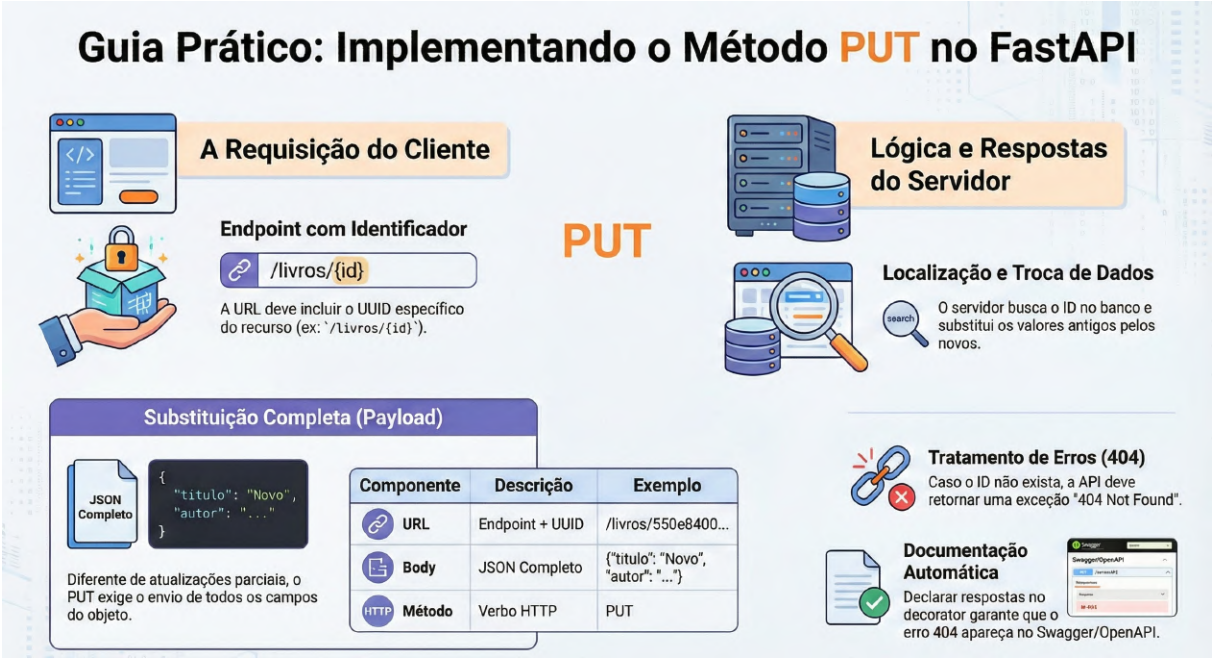
Dica Didática: Note que, embora o banco de dados seja apenas um dicionário em memória, o fluxo de comunicação HTTP é idêntico ao de um sistema profissional. Nos próximos capítulos, aprenderemos como atualizar esses registros usando os métodos PUT e PATCH.

6. O Método PUT – Substituição Total de Registros

Neste capítulo, avançaremos para o método **PUT**. Enquanto o método POST cria um novo recurso, o PUT é utilizado para **substituir integralmente** um registro já existente na base de dados. Veremos como identificar o recurso correto via URL e como garantir que a atualização ocorra de forma segura e bem documentada.

O Conceito do Método PUT

Diferente do POST, onde enviamos apenas os dados do livro, o PUT exige duas informações fundamentais:



1. **Identificação na URL:** O UUID do livro que desejamos modificar (ex: `/livros/{livro_id}`).
2. **Corpo da Requisição (JSON):** Uma versão completa e atualizada do objeto. No PUT, espera-se que todos os campos (autor, título, editora e ano) sejam enviados para substituir os valores antigos.

Se a operação for bem-sucedida, o servidor processa a substituição e retorna o código **200 OK** para o cliente.

Implementação no Servidor (`api.py`)

No FastAPI, utilizaremos o decorador `@app.put`. A função precisará receber o `livro_id` (vido da URL) e os novos dados (vindos do corpo da requisição).

Abaixo, a implementação considerando nossa base de dados em memória:

```
@app.put("/livros/{livro_id}", response_model=Livro, responses={404: {"description": "Livro não
↪  encontrado"}})
async def atualizar_livro(livro_id: UUID, livro_update: LivroPostPut) -> Livro:
    # Percorremos a base de dados para encontrar o livro pelo UUID
    for index, livro in livros_db.items():
        if livro["id"] == livro_id:
            # Substituímos na base de dados
            livros_db[index] = dict(
                uuid = livro_id,
                autor = livro_update.autor,
                titulo = livro_update.titulo,
                editora = livro_update.editora,
                ano = livro_update.ano
            )

            # Retornamos o objeto convertido em modelo Livro
            return Livro(**livro_atualizado)

    # Caso o loop termine sem encontrar o ID, lançamos o erro 404
    raise HTTPException(status_code=404, detail="Livro não encontrado")
```

Implementação no Cliente (`cliente.py`)

Agora, atualizaremos o nosso script de cliente para suportar a atualização completa de um livro. Esta função solicitará o UUID e, em seguida, os novos dados.

```
def atualizar_livro_inteiro():
    print("\nAtualização Completa (PUT)")
    uuid_livro = input("Digite o UUID do livro a atualizar: ").strip()

    print("\nDigite os NOVOS dados completos do livro:")
    autor = input("Novo Autor: ")
    titulo = input("Novo Título: ")
    editora = input("Nova Editora: ")
    ano = int(input("Novo Ano: "))

    payload = {
        "autor": autor,
        "titulo": titulo,
        "editora": editora,
        "ano": ano
    }

    # Realizamos a requisição PUT concatenando o ID na URL
```

```
    resp = requests.put(f"{API_URL}/livros/{uuid_livro}", json=payload)
    print("\nLivro Atualizado:")
    tratar_resp(resp)
```

Atualizando o Menu do Cliente

Para que a nova funcionalidade seja acessível, adicione a opção correspondente ao menu principal:

```
def menu():
    while True:
        # ... outras opções
        print("3. Adicionar novo livro")
        print("4. Atualizar livro inteiro (PUT)")
        print("0. Sair")

        opcao = input("\nEscolha uma opção: ").strip()

        if opcao == "1":
            listar_livros()
        elif opcao == "2":
            obter_livro_por_id()
        elif opcao == "3":
            adicionar_livro()
        elif opcao == "4":
            atualizar_livro_inteiro()
        elif opcao == "0":
            break
```

6.5 Testando a Substituição

Para testar, siga este fluxo:

1. Liste os livros (Opção 1) e copie um UUID existente.
2. Escolha a Opção 4 e cole o UUID.
3. Insira dados diferentes (por exemplo, altere a editora para “Nova Editora”).
4. Liste novamente os livros e confirme que o registro com aquele UUID agora possui as novas informações, sem ter criado um novo ID.

Neste estágio, você já domina a criação, leitura e substituição de dados. No próximo capítulo, exploraremos o método **PATCH**, que permite atualizações parciais de forma mais eficiente.

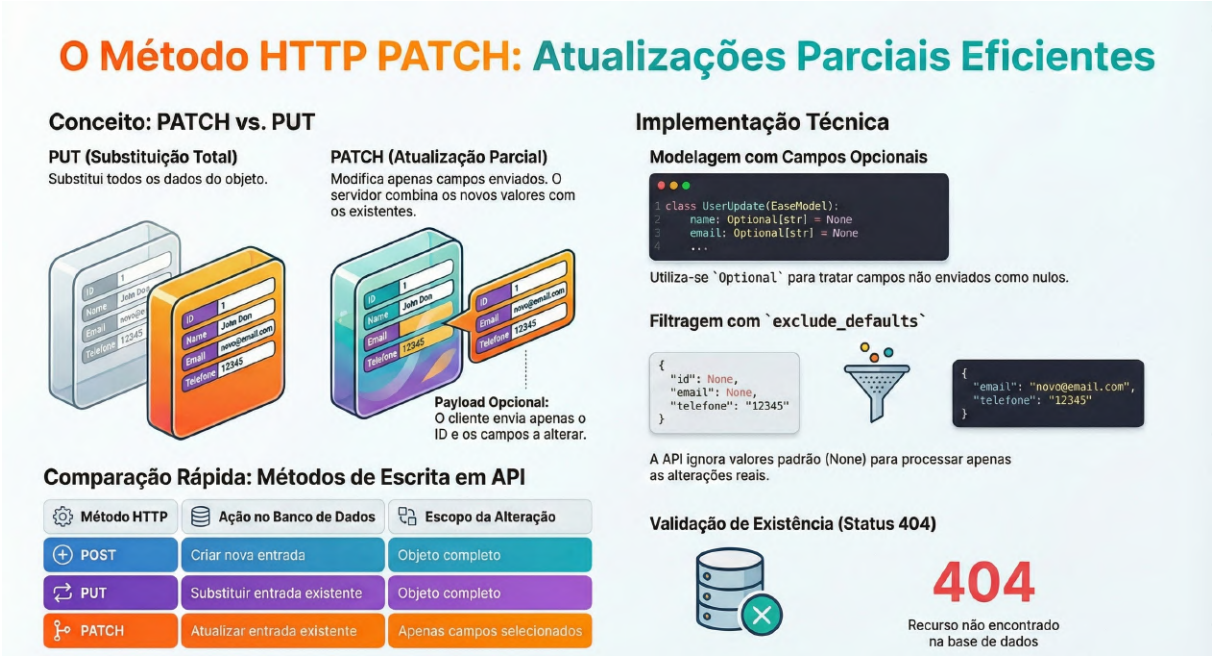
O método PUT garante a integridade do seu registro ao substituí-lo por uma versão nova e completa!

7. O Método PATCH – Realizando Atualizações Parciais

Neste capítulo, exploraremos o método **PATCH**, uma alternativa mais eficiente ao PUT quando desejamos modificar apenas algumas informações de um registro, sem a necessidade de reenviar todos os dados para o servidor. Você aprenderá a criar modelos com campos opcionais e a implementar a lógica de "mesclagem" de dados no back-end e no front-end.

PATCH vs. PUT: Qual a diferença?

Embora ambos os métodos sirvam para atualizar dados, a intenção semântica deles no protocolo HTTP é diferente:



• **PUT:** Realiza uma **substituição total**. O cliente deve enviar o objeto completo; caso algum campo seja omitido, ele pode ser apagado ou causar erro.

• **PATCH:** Realiza uma **substituição parcial**. O cliente envia apenas os campos que deseja alterar, e o servidor preserva os demais dados inalterados.

Modelagem de Dados com Campos Opcionais

Para que o FastAPI aceite atualizações parciais, não podemos usar os modelos anteriores (`Livro` ou `LivroPostPut`), pois eles exigem todos os campos. Precisamos de um novo modelo onde todos os atributos sejam opcionais.

Utilizamos a classe `Optional` da biblioteca `typing` e definimos o valor padrão como `None`. Veja a implementação no arquivo `api.py`:

```
from typing import List, Optional

class LivroPatch(BaseModel):
    autor: Optional[str] = None
    titulo: Optional[str] = None
    editora: Optional[str] = None
    ano: Optional[int] = None
```

Ao definir o valor como `None`, o Pydantic entende que, se o campo não for enviado no JSON, ele não deve gerar erro de validação.

Implementação no Servidor (`api.py`)

A lógica do PATCH no servidor exige um cuidado especial: precisamos filtrar quais campos o usuário realmente enviou para não sobrescrever os dados existentes com `None` acidentalmente.

```
@app.patch("/livros/{livro_id}", response_model=Livro, responses={404: {"description": "Livro
↪  não encontrado"}})
async def atualizar_parcial(livro_id: UUID, livro_update: LivroPatch) -> Livro:
    # 1. Busca o livro na base de dados
    for index, livro in livros_db.items():
        if livro["uuid"] == livro_id:
            # 2. Converte o modelo em dicionário, excluindo campos não enviados
            for key, value in livro_update.model_dump(exclude_defaults=True).items():
            livro[key] = value

            return Livro(**livro_db[index])

    raise HTTPException(status_code=404, detail="Livro não encontrado")
```

O método `model_dump(exclude_defaults=True)` é fundamental, pois ele garante que os campos com valor padrão `None` sejam excluídos.

Implementação no Cliente (`cliente.py`)

No cliente, criaremos uma interface que pergunta cada campo ao usuário. Se ele apenas pressionar **ENTER**, o campo será ignorado na atualização.

```
def atualizar_parcial():
    print("\nAtualização Parcial (PATCH)")
    uuid_livro = input("Digite o UUID do livro: ").strip()

    print("Digite os campos que deseja alterar (deixe em BRANCO para não alterar):")
    autor = input("Novo Autor: ").strip()
    titulo = input("Novo Título: ").strip()
```

```
    editora = input("Nova Editora: ").strip()
    ano = input("Novo Ano: ").strip()

    # Montamos o payload apenas com o que foi preenchido
    payload = {}

    if autor: payload["autor"] = autor
    if editora: payload["editora"] = editora
    if titulo: payload["titulo"] = titulo
    if ano: payload["ano"] = int(ano)

    if not payload:
        print("Nenhuma alteração informada.")
        return

    resp = requests.patch(f"{API_URL}/livros/{uuid_livro}", json=payload)
    print("\nResultado da atualização parcial:")
    tratar_resp(resp)
```

Atualizando o Menu e Testando

Adicione a nova opção ao menu do seu `cliente.py`:

```
# No menu()
print("5.Atualizar parcialmente (PATCH)")

# No loop do menu
elif opcao == "5":
    atualizar_parcial()
```

Como testar:

1. Inicie a API e liste os livros para obter um UUID.
2. No cliente, escolha a opção 5.
3. Informe o UUID e, quando solicitado o “Novo Autor”, digite um nome. Nos outros campos, apenas aperte **ENTER**.
4. Ao listar novamente, você verá que apenas o autor mudou, enquanto o título, editora e ano permanecem os mesmos.

Com o método PATCH, sua API torna-se mais profissional e economiza recursos, transmitindo apenas o necessário pela rede. No próximo capítulo, aprenderemos como remover registros definitivamente com o método **DELETE**.

8. O Método DELETE – Removendo Registros com Segurança

Neste capítulo final da nossa base de operações de CRUD (*Create, Read, Update, Delete*), aprenderemos a implementar a funcionalidade de exclusão. O método **DELETE** é utilizado para remover permanentemente um registro da nossa base de dados. Diferente dos métodos POST e PUT, ele não exige o envio de um corpo de mensagem (JSON), necessitando apenas da identificação correta do recurso na URL.

O Modelo de Resposta para Confirmação

Embora o protocolo HTTP permita retornar o status **204 (No Content)**, que indica sucesso sem retornar dados, é uma boa prática didática retornar uma mensagem de confirmação para que o usuário ou o sistema cliente saiba exatamente o que foi removido.

Para isso, definiremos um modelo específico chamado `ConfirmaDelete` no arquivo `api.py`:

```
class ConfirmaDelete(BaseModel):
    mensagem: str
    uuid: UUID
```

Implementação no Servidor (`api.py`)

A função de exclusão seguirá uma lógica similar à busca por ID: localizamos o registro e, caso ele exista, utilizamos o comando `del` para removê-lo do nosso dicionário em memória.

```
@app.delete("/livros/{livro_id}", response_model=ConfirmaDelete, responses={404:
↪  {"description": "Livro não encontrado"}})
async def deletar_livro(livro_id: UUID) -> ConfirmaDelete:
    # Percorre a base de dados para encontrar o índice do livro
    for index, livro in livros_db.items():
        if livro["uuid"] == livro_id:

            # Remove o registro do dicionário
            del livros_db[index]

            # Retorna a confirmação com o título e o ID
            return ConfirmaDelete(
                mensagem=f"Livro '{livro_id}' deletado com sucesso",
                uuid=livro_id
            )

    # Se não encontrar o ID, retorna 404
    raise HTTPException(status_code=404, detail="Livro não encontrado")
```

Detalhes Técnicos Importantes:

• **Identificação:** O `livro_id` capturado na URL é o que garante que não deletaremos o registro errado.

• **Controle de Erro:** Caso o cliente tente deletar um ID que já foi removido ou que nunca existiu, o servidor responderá com **404 Not Found**, mantendo a integridade da comunicação.

Implementação no Cliente (`cliente.py`)

No lado do cliente, a implementação é direta e rápida. Precisamos apenas capturar o UUID do livro e disparar a requisição DELETE.

```
def deletar_livro_cliente():
    print("\nExclusão de Registro (DELETE)")
    uuid_livro = input("Digite o UUID do livro a ser deletado: ").strip()

    # Realiza a requisição DELETE (sem necessidade de payload)
    resp = requests.delete(f"{API_URL}/livros/{uuid_livro}")

    print("\nResultado da exclusão:")
    tratar_resp(resp)
```

Atualização Final do Menu

Para encerrar nossa aplicação cliente, adicionaremos a sexta opção ao menu interativo:

```
def menu():
    while True:
        # ... outras opções
        print("5. Atualizar parcialmente (PATCH)")
        print("6. Deletar livro (DELETE)")
        print("0. Sair")

        opcao = input("\nEscolha uma opção: ").strip()

        if opcao == "1":
            listar_livros()
        # ... outras condicionais
        elif opcao == "6":
            deletar_livro_cliente()
        elif opcao == "0":
            break
```

Testando o Fluxo de Exclusão

1. **Liste os livros** para copiar um UUID existente.
2. **Execute a opção 6** e cole o UUID escolhido.

3. **Verifique a mensagem** de sucesso retornada pelo servidor (Status 200).
4. **Tente deletar o mesmo UUID novamente**: você verá que o sistema agora retorna um erro **404**, pois o livro não consta mais no banco de dados.

Conclusão do Módulo Base

Neste ponto, você construiu uma API completa e um cliente funcional capaz de realizar todas as operações essenciais de um sistema de back-end. Aprendemos a lidar com diferentes métodos HTTP, validação de dados com Pydantic e tratamento de erros profissionais.

Até agora, nossos dados estão "em memória" e são perdidos sempre que o servidor reinicia. No próximo módulo, aprenderemos a dar o próximo passo: a **persistência real**, utilizando bancos de dados SQL para que as informações sejam salvas permanentemente.

9.Persistência e Ferramentas – Instalando o DBeaver

Até este ponto, nossa API funcionou utilizando uma base de dados em memória. Isso significa que, toda vez que o servidor era reiniciado, os dados eram perdidos. Para evoluirmos para um sistema profissional, precisamos de **persistência de dados**, salvando as informações em um banco de dados real, como o **SQLite**.

Neste capítulo, aprenderemos a instalar e configurar o **DBeaver**, uma ferramenta essencial para visualizar e gerenciar nossos dados de forma gráfica e intuitiva.

Por que utilizar o DBeaver?

O DBeaver é uma ferramenta de administração de banco de dados universal e gratuita (na sua edição *Community*). Ela nos permitirá abrir o arquivo do banco de dados que nossa API criará e visualizar as tabelas de livros, os registros e os tipos de dados sem precisar digitar comandos complexos de SQL no terminal.

Instalando o DBeaver



Instalação no Windows

Para usuários de Windows, o processo segue o padrão de instaladores executáveis:

1. **Download:** Acesse o site oficial https://dbeaver.io/ e vá até a seção de *Download*. Escolha a versão **Community Edition**.
2. **Execução:** Abra o instalador baixado.
3. **Configuração:**

◦ Selecione o idioma (Português).

◦ Aceite os termos da licença.

◦ Escolha se a instalação será para todos os usuários ou apenas para você.

◦ Siga as instruções clicando em "Próximo" até concluir a instalação.

Ao final, um atalho será criado na sua área de trabalho para fácil acesso.

Instalação no Linux (Base Debian/Ubuntu)

Para sistemas baseados em Linux, como o Ubuntu, o processo utiliza pacotes `.deb`:

1. **Download:** No site oficial, baixe o pacote **Linux Debian**.
2. **Terminal:** Abra o terminal na pasta onde o arquivo foi baixado e utilize o comando de instalação:

```
sudo apt install ./NOME_DO_PACOTE_BAIXADO.deb
```

(Nota: substitua o nome do arquivo pelo nome exato do pacote baixado).

4. **Acesso:** Após a conclusão, o DBeaver estará disponível no seu menu de aplicativos, geralmente na seção de "Desenvolvimento".

Primeira Conexão com SQLite

Uma vez instalado, o DBeaver está pronto para se conectar ao banco de dados que utilizaremos com o FastAPI. Embora a criação do banco ocorra via código (que veremos nos próximos capítulos), o processo de conexão no DBeaver segue estes passos:

1. Clique em **Nova Conexão** (ícone de tomada com um sinal de mais).
2. Selecione **SQLite** na lista de bancos de dados.
3. Indique o caminho do arquivo (ex: `database.db`) no seu computador.

4. **Teste a conexão:** Na primeira vez, o DBeaver pode solicitar o download dos drivers do SQLite. Clique em “Baixar” e aguarde a confirmação de sucesso.

Com o DBeaver configurado, você terá uma visão clara da estrutura da sua API, facilitando testes e correções durante o desenvolvimento. Agora estamos prontos para integrar o banco de dados ao nosso código Python utilizando o **SQLModel**.

O DBeaver será sua janela para entender como os dados estão sendo realmente gravados!

10. Persistência de Dados com SQLModel e Refatoração do Projeto

Neste capítulo, daremos um passo fundamental para transformar nossa aplicação em um sistema profissional. Até agora, nossa API utilizava o armazenamento em memória, o que causava a perda de todas as informações sempre que o servidor era reiniciado. Para solucionar essa volatilidade, introduziremos a **persistência de dados** e aprenderemos a organizar nosso código de forma modular e escalável.

O Papel do SQLModel e o Conceito de ORM

Para gerenciar a comunicação entre o Python e o banco de dados, utilizaremos a biblioteca **SQLModel**. Ela atua como um **ORM** (*Object-Relational Mapping*), que é o responsável por traduzir os objetos que criamos no Python para a linguagem relacional do SQL (como o SQLite ou PostgreSQL).

O SQLModel é especialmente poderoso porque utiliza o Pydantic internamente, permitindo que usemos os mesmos modelos tanto para a validação de dados da API quanto para a definição das tabelas no banco de dados.

Preparação do Ambiente Antes de iniciarmos a codificação, precisamos instalar a biblioteca e organizar as dependências do projeto:

```
# Instalação do SQLModel
pip install sqlmodel

# Opcional: Instalação de tipos para o VS Code reconhecer o Requests
pip install types-requests

# Gerando o arquivo de requisitos
pip freeze > requirements.txt
```

Reestruturação do Projeto

À medida que a aplicação cresce, manter todo o código em um único arquivo torna-se inviável. Por isso, adotaremos uma estrutura de pastas profissional:

• **/cliente:** Contém o arquivo `cliente.py` e um arquivo `__init__.py` para tratá-lo como um módulo.

• **/api:** Pasta principal da aplicação, contendo:

  ◦ `__init__.py`: Identificador de módulo.

  ◦ `models.py`: Definição de todos os esquemas e tabelas do SQLModel.

- `api.py`: Arquivo principal (que futuramente será renomeado ou refatorado).

Modelagem Avançada em `models.py`

Em vez de criar classes isoladas, utilizaremos a **herança de classes** para evitar repetição de código e separar as responsabilidades de cada modelo (o que entra no banco, o que sai na API e o que é usado para atualizações).

O Modelo Base e a Tabela Principal Criamos uma classe base com os campos comuns e uma classe específica que representa a tabela no banco de dados:

```
from sqlmodel import SQLModel, Field
from uuid import UUID, uuid4
from typing import Optional

# Classe base com campos compartilhados
class LivroBase(SQLModel):
    autor: str = Field(index=True)
    titulo: str = Field(index=True)
    editora: str = Field(index=True)
    ano: int = Field(index=True)

# Classe que define a tabela real no SQLite
class Livro(LivroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)
    uuid: UUID = Field(default_factory=uuid4, unique=True)
```

O campo `id` (inteiro) é a chave primária interna do banco, enquanto o `uuid` (UUID) é o identificador único que expomos para o mundo exterior.

A classe `Livro` herda os campos de `LivroBase` e é definido como uma tabela do banco de dados. Isso equivale a criar uma tabela com todos campos de `LivroBase` e `Livro` juntos, sem precisar repetir os campos em cada um das classes.

O `default_factory` informa ao **SQLModel** que, ao criar o campo `uuid`, o valor padrão **não é fixo**, mas sim gerado dinamicamente por uma **função**. Cada vez que um novo objeto é criado, essa função é chamada automaticamente para produzir um valor único para o campo.

Modelos Especializados para a API Para garantir uma boa prática de segurança e clareza, definimos modelos para cada tipo de interação:

1. **LivroResposta:** Usado para retornar dados ao cliente, ocultando o `id` interno do banco e exibindo apenas o `uuid`.

2. **LivroPost e LivroPut:** Modelos simples que herdam de `LivroBase` para criação e substituição total de registros.
3. **LivroPatch:** Modelo para permitir atualizações parciais.
4. **ConfirmaDelete:** Modelo para confirmação de exclusão

```
# Modelo para resposta da API
class LivroResposta(LivroBase):
    uuid: UUID

# Modelo para criação (POST)
class LivroPost(LivroBase):
    ...

# Modelo para criação (PUT)
class LivroPut(LivroBase):
    ...

# Modelo para atualização parcial (PATCH)
class LivroPatch(SQLModel):
    autor: str | None = None
    titulo: str | None = None
    editora: str | None = None
    ano: int | None = None

# Modelo para deletar (DELETE)
class ConfirmaDelete(BaseModel):
    mensagem: str
    uuid:UUID
```

Quando usamos `str | None`, isso equivale a dizer ao Python que o campo pode uma `string` ou `None`. É o mesmo que utilizar o `Optional`, que foi usado anteriormente. ## Por que separar os modelos?

A separação (Resposta, Post, Put, Patch) é considerada uma **boa prática de desenvolvimento**. Isso permite que:

- O usuário da API não precise conhecer detalhes internos do banco (como o ID numérico).
- O processo de criação (POST) não exija que o cliente envie um UUID, já que o servidor deve gerá-lo automaticamente.
- O processo de atualização parcial (PATCH) aceite campos nulos ou ausentes sem causar erros de validação.

Com essa estrutura robusta de modelos, estamos prontos para configurar a conexão real com o banco de dados e realizar as primeiras operações de escrita persistente.

No próximo capítulo, aprenderemos a configurar o motor do banco de dados e a criar as tabelas automaticamente!

11. Persistência Real – Criando a Base de Dados e o Ciclo de Vida da API

Neste capítulo, daremos o passo final para tirar nossa aplicação da memória volátil e levá-la para o armazenamento permanente. Vamos configurar o "motor" (engine) do banco de dados, criar as tabelas automaticamente e entender como o FastAPI gerencia tarefas importantes antes de começar a receber requisições.

Configurando o Motor do Banco de Dados (`database.py`)

Para que o **SQLModel** saiba onde e como salvar os dados, precisamos criar um arquivo central de conexão chamado `database.py` dentro da pasta `/api`. Este arquivo será responsável por definir o caminho do banco de dados SQLite e gerenciar as sessões de comunicação.

Definição do Caminho e da Engine Utilizaremos a biblioteca `pathlib` para garantir que o arquivo do banco de dados seja criado no local correto, independentemente do sistema operacional.

```
from sqlmodel import create_engine, SQLModel, Session
from pathlib import Path

from api.models import Livro # Importamos o modelo que deve virar tabela

# Define o caminho do arquivo do banco de dados
DATABASE_PATH = Path("database.db")
DATABASE_URL = f"sqlite:///{DATABASE_PATH}"

# Argumento necessário especificamente para o SQLite
CONNECT_ARGS = {"check_same_thread": False}

# Criação da Engine (o motor de conexão)
engine = create_engine(DATABASE_URL, connect_args=CONNECT_ARGS)
```

`check_same_thread=False`: No SQLite, este argumento é essencial para permitir que o FastAPI trabalhe de forma assíncrona, permitindo que diferentes requisições acessem o banco ao mesmo tempo.

Funções Utilitárias Precisamos de duas funções fundamentais: uma para criar as tabelas e outra para abrir sessões de conversa com o banco.

```
def criar_db_tabelas():
    # Esta função lê todos os modelos que possuem "table=True" e cria as tabelas no arquivo .db
    SQLModel.metadata.create_all(engine)

def get_session():
    # Função para liberar uma sessão de conexão que será usada nos endpoints
```

```
    with Session(engine) as session:
        yield session
```

O Que a Palavra-Chave `yield` faz em Python? A palavra-chave `yield` em Python é usada para criar um gerador que pode produzir uma sequência de valores para iteração. Quando uma função contendo `yield` é chamada, ela retorna um objeto gerador que é então iterado para recuperar valores um de cada vez. Isso é particularmente útil para eficiência de memória e ao lidar com sequências grandes ou potencialmente infinitas.

Com o `yield`, você pode escrever códigos Python mais eficientes que lidam com grandes conjuntos de dados com facilidade.

Integrando ao FastAPI com o Ciclo de Vida (`main.py`)

Agora que temos o banco configurado, precisamos dizer ao FastAPI para executar a criação das tabelas assim que o servidor iniciar. Para isso, utilizamos um conceito moderno chamado **Lifespan** (Ciclo de Vida).

O **Lifespan** é um gerenciador de contexto assíncrono que permite executar códigos antes da API ficar disponível para o público (como criar tabelas ou carregar dados) e após ela ser encerrada.

Vamos criar na pasta `/api`, um arquivo chamado `main.py`:

```
from fastapi import FastAPI
from contextlib import asynccontextmanager
from api.database import criar_db_tabelas

@asynccontextmanager
async def lifespan(app: FastAPI):
    # Tudo antes do "yield" acontece no início da API
    criar_db_tabelas()
    yield
    # Tudo depois do "yield" acontece no encerramento da API

app = FastAPI(title="API de Livros", lifespan=lifespan)
```

A partir desse ponto não vamos mais usar o arquivo `api.py`, pois o `main.py` agora é responsável por rodar a nossa API.

Verificando a Estrutura com DBeaver

Ao rodar a aplicação pela primeira vez com o comando `fastapi dev api/main.py`, o arquivo `database.db` será gerado automaticamente. Para garantir que tudo deu certo, podemos usar o **DBeaver** para inspecionar o interior do arquivo.

Passo a Passo da Conexão

1. **Inicie o DBeaver:** Abra o programa através do atalho criado na sua área de trabalho ou menu de aplicativos.
2. **Crie uma Nova Conexão:** Clique em **Database** e em **New Database Connection**.
3. **Selecione o Banco de Dados:** Na lista de opções que aparecerá, selecione **SQLite** e clique em "Próximo".
4. **Localize o Arquivo:** Na tela de configuração, clique em "Open" e navegue até a pasta do seu projeto para selecionar o arquivo `database.db`.
5. **Teste a Conexão:** Clique no botão **"Testar conexão"**. Caso seja a primeira vez que você utiliza o SQLite no DBeaver, o programa solicitará o download dos drivers necessários; clique em "Baixar" e aguarde a confirmação de sucesso.
6. **Finalize:** Clique em "Finalizar" para concluir a configuração.

Ao conectar o DBeaver ao arquivo `database.db`, você notará que a tabela **livros** foi criada com as seguintes especificações automáticas:

• **Colunas:** Autor (varchar), Título (varchar), Editora (varchar), Ano (integer).

• **Identificadores:** O campo `id` como chave primária inteira e o `uid` como texto único.

A grande vantagem de utilizar um ORM como o SQLModel é que essa mesma definição de código funcionaria da mesma forma se estivéssemos utilizando bancos mais robustos, como o PostgreSQL.

Agora que a estrutura física do seu banco de dados está pronta, no próximo capítulo aprenderemos a realizar as primeiras operações de inserção e busca real!.

12. Refatoração e Organização de Rotas com APIRouter

Com o crescimento da aplicação e a migração para um banco de dados real, manter todo o código em um único arquivo torna-se ineficiente. Neste capítulo, aprenderemos a organizar nossa API de forma modular, utilizando o **APIRouter** para gerenciar as rotas e implementando a lógica de busca de dados diretamente do banco de dados SQLite utilizando o **SQLModel**.

Modularização com APIRouter

À medida que adicionamos mais funcionalidades, a boa prática recomenda separar os *endpoints* por contexto. Para isso, criaremos uma pasta chamada `routers` e nela o arquivo `livros_router.py`.

Diferente do que vínhamos fazendo, não utilizaremos a instância global `app` para definir as rotas. Em vez disso, utilizaremos o **APIRouter**, que permite agrupar rotas sob um mesmo prefixo e etiquetas de documentação.

```
from fastapi import APIRouter
from api.models import Livro, LivroResposta

# Criando a rota com prefixo e tags para organização no Swagger
router = APIRouter(prefix="/livros", tags=["livros"])

@router.get("/", response_model=list[LivroResposta])
async def listar_livros():
    # A implementação agora buscará os dados no banco real
    pass
```

Ao definir um prefixo como `"/livros"`, não precisamos repetir o nome do recurso no caminho do método; basta usar `"/"`.

Injeção de Dependência de Sessão

Para interagir com o banco de dados, cada função da nossa API precisa de uma sessão de conexão ativa. De acordo com as fontes, utilizaremos o conceito de **SessionDep** (Dependência de Sessão), que injeta automaticamente uma sessão do banco de dados na função sempre que ela é chamada.

Para isso, definimos um tipo anotado que utiliza a nossa função `get_session` (criada no capítulo anterior):

```
from typing import Annotated
from fastapi import APIRouter, Depends
from sqlmodel import Session
from api.database import get_session
```

```
# Criando um apelido para a dependência de sessão
SessionDep = Annotated[Session, Depends(get_session)]
```

Implementando a Busca com SQLModel

Agora, podemos refatorar a função de listagem para realizar uma consulta SQL real em vez de ler um dicionário em memória. Utilizaremos o método `select` do SQLModel para buscar todos os registros da tabela `Livro`.

```
from sqlmodel import select

@router.get("/", response_model=list[LivroResposta])
async def listar_livros(session: SessionDep) -> list[LivroResposta]:
    # Executa a query: SELECT * FROM livro
    livros = session.exec(select(Livro)).all()

    # Converte os objetos 'Livro' (do banco) para 'LivroResposta' (da API)
    # Isso garante que o ID interno do banco não seja exposto
    return [LivroResposta.model_validate(livro) for livro in livros]
```

O método `model_validate` do Pydantic é essencial aqui, pois ele filtra o objeto original do banco de dados e mantém apenas os campos definidos no modelo de resposta, como o **UUID**.

Registro das Rotas no Arquivo Principal

Para que o FastAPI reconheça as novas rotas organizadas, precisamos registrá-las no arquivo `main.py` utilizando o comando `include_router`.

```
from fastapi import FastAPI
from api.routers import livros_router

app = FastAPI(title="API de Livros")

# Registrando o módulo de rotas de livros
app.include_router(livros_router.router)
```

Dados Iniciais para Testes (Default Data)

Como deletamos nosso dicionário em memória e o banco de dados inicia vazio, devemos criar uma função temporária para popular o banco com dados padrão assim que a API inicia. Isso nos permite testar se a listagem está funcionando corretamente.

No arquivo `database.py`, podemos implementar uma lógica que verifica se a tabela está vazia e, caso esteja, insere livros com UUIDs fixos.

```
from uuid import UUID
from sqlmodel import select

livros_default_data = {
    1: {
    "id": UUID(),
    "autor": "George Orwell",
    "titulo": "1984",
    "editora": "Companhia das Letras",
    "ano": 1949
    },
    2: {
    "id": UUID(),
    "autor": "J.R.R. Tolkien",
    "titulo": "O Hobbit",
    "editora": "HarperCollins",
    "ano": 1937
    }
}

def adicionar_livros_default():
    with Session(engine) as session:
        # Verifica se já existem livros no banco
        existe = session.exec(select(Livro)).first()

        if not existe:
            # Criação de dados fictícios para teste
            for data in livros_default_data:
                livro = Livro(
                    uuid=data.get("uuid"),
                    autor=data.get("autor"),
                    titulo=data.get("titulo"),
                    editora=data.get("editora"),
                    ano=data.get("ano")
                )
                session.add(livro)
                session.commit()
```

Esta função deve ser chamada dentro do nosso gerenciador de ciclo de vida (**lifespan**) no arquivo `main.py`, garantindo que o banco de dados esteja sempre pronto para uso no momento em que o servidor sobe.

```
from api.database import criar_db_tabelas, adicionar_livros_default

@asynccontextmanager
async def lifespan(app: FastAPI):
    criar_db_tabelas()
    adicionar_livros_default()
    yield

app = FastAPI(title="API de Livros", lifespan=lifespan)
```

Agora, sua API tem uma estrutura profissional: rotas organizadas em arquivos separados, injeção de

dependência para o banco de dados e uma estratégia de persistência que mantém seus dados seguros mesmo após reiniciar o servidor.

13. Consultas Individuais e Criação de Dados com SQLModel

Neste capítulo, avançaremos na implementação dos nossos endpoints, migrando a lógica de busca individual e de criação de registros do antigo modelo em memória para o sistema de persistência com **SQLModel**. Aprenderemos a utilizar operadores avançados do Python e a garantir que os dados sejam devidamente gravados no banco de dados SQLite.

Recuperando um Registro Específico (GET por UUID)

Embora já tenhamos a listagem geral, é fundamental que uma API permita consultar os detalhes de um único item. Para isso, utilizaremos o **UUID** como filtro na nossa consulta SQL.

O Operador Morsa (Walrus Operator) Para tornar o código mais limpo, utilizaremos o operador de atribuição e comparação `:=` (conhecido como *Walrus Operator*). Ele nos permite atribuir o resultado da busca a uma variável e, simultaneamente, verificar se esse resultado é válido dentro de uma estrutura condicional `if`.

Implementação do Endpoint No arquivo `livros_router.py`, a implementação segue esta lógica:

```
from uuid import UUID
from sqlmodel import Session, select
from api.models import Livro, LivroResposta
from fastapi import HTTPException

@router.get("/{livro_id}", response_model=LivroResposta, responses={404: {"description": "Livro
↪  não encontrado"}})
async def obter_livro(livro_id: UUID, session: SessionDep) -> LivroResposta:
    # O operador := atribui o resultado a 'livro' e testa se não é None
    if (livro := session.exec(select(Livro).where(Livro.uuid == livro_id)).first()):
        return LivroResposta.model_validate(livro)

    # Caso a busca retorne None, levantamos a exceção 404
    raise HTTPException(status_code=404, detail="Livro não encontrado")
```

O método `.where(Livro.uuid == livro_id)` é a tradução do comando SQL `SELECT * FROM livro WHERE uuid = ...`. O método `.first()` garante que pegaremos apenas a primeira ocorrência encontrada.

O Método POST com Persistência Real

Agora, modificaremos o método de criação de livros. Ao contrário da versão em memória, onde apenas adicionávamos um item a um dicionário, aqui utilizaremos o ciclo de **Adição e Confirmação (Commit)** do banco de dados.

O Fluxo de Gravação

1. **Recebimento:** A API recebe um objeto do tipo `LivroPost` (que contém apenas autor, título, editora e ano).
2. **Instanciação:** Criamos um objeto da classe `Livro` (o modelo que representa a tabela), preenchendo seus campos.
3. **Sessão:** Adicionamos o objeto à sessão com `session.add()`.
4. **Persistência:** Confirmamos a gravação com `session.commit()`. Sem o *commit*, os dados não são salvos permanentemente no arquivo `.db`.

```
@router.post("/", response_model=LivroResposta)
async def adicionar_livro(livro: LivroPost, session: SessionDep) -> LivroResposta:
    # Gera um novo uuid
    novo_uuid = uuid4()

    # Criando a instância que será gravada no banco
    livro_gravado = Livro(
        uuid=novo_uuid,
        autor=livro.autor,
        titulo=livro.titulo,
        editora=livro.editora,
        ano=livro.ano
    )

    session.add(livro_gravado) # Adiciona à fila de gravação
    session.commit()           # Grava definitivamente no banco

    # Retornamos os dados validados no modelo de resposta
    return LivroResposta.model_validate(livro_gravado)
```

Validando a Persistência no Banco de Dados

Para garantir que a "mágica" da persistência está ocorrendo, não devemos confiar apenas no retorno da API. É essencial verificar o arquivo físico do banco de dados.

Utilizando o **DBeaver**, você pode abrir um editor SQL (**SQL Editor -> Open SQL Script ou apertar a tecla F3**) e executar o seguinte comando:

```
SELECT * FROM livro;
```

Ao executar esta consulta, você deverá ver todos os registros, incluindo aqueles adicionados via método POST, com seus respectivos IDs internos (inteiros) e UUIDs (identificadores universais) devidamente preenchidos.

Dica Didática: Note que, ao usar `model_validate`, a API filtra automaticamente o `id` interno do banco, entregando ao cliente apenas o que definimos no `LivroResposta`. Isso mantém a estrutura interna do seu banco de dados protegida e organizada.

No próximo capítulo, finalizaremos o ciclo CRUD implementando as atualizações (PUT e PATCH) e a exclusão (DELETE) com SQLModel.

14. Finalizando o CRUD – PUT, PATCH e DELETE com SQLModel

Neste capítulo, completaremos o ciclo de operações da nossa API migrando os métodos de atualização e exclusão para o **SQLModel**. A grande vantagem dessa etapa é observar como a persistência em banco de dados simplifica a lógica de busca e manipulação de registros em comparação ao uso de dicionários em memória.

O Método PUT: Substituição Integral

O método **PUT** é utilizado para atualizar um livro por completo. Diferente do POST, ele exige o identificador (**UUID**) na URL e os novos dados no corpo da requisição.

Implementação no Servidor Para atualizar o registro, primeiro localizamos o livro no banco de dados através do UID. Em seguida, percorremos os atributos do modelo de atualização e os aplicamos ao objeto encontrado.

```
@router.put("/{livro_id}", response_model=LivroResposta, responses={404: {"description": "Livro
↪  não encontrado"}})
async def atualizar_livro(livro_id: UUID, livro_update: LivroPut, session: SessionDep) ->
↪  LivroResposta:
    # 1. Busca o livro pela identidade (UUID)
    if (livro := session.exec(select(Livro).where(Livro.uuid == livro_id)).first()):
        # 2. Atualiza os atributos de forma automática
        for key, value in livro_update:
            setattr(livro, key, value)

        # 3. Salva e atualiza a instância com os dados do banco
        session.add(livro)
        session.commit()
        session.refresh(livro) # Garante que temos a versão final do banco
        return LivroResposta.model_validate(livro)

    raise HTTPException(status_code=404, detail="Livro não encontrado")
```

Ao usar `session.add(livro)` em um objeto que já possui um ID, o SQLModel entende que deve realizar uma substituição de valores, preservando os identificadores internos.

`setattr()` é uma função do python que define dinamicamente o valor de um atributo de um objeto, sendo equivalente a usar `autor = livro.autor`, para cada atributo do nosso modelo `Livro`.

O Método PATCH: Atualização Parcial e Filtros

O **PATCH** é mais complexo que o PUT porque permite que o cliente envie apenas um ou alguns campos. Para que isso funcione sem "apagar" os dados existentes com valores nulos, precisamos filtrar a

entrada.

Filtrando Dados com `model_dump` Utilizamos o método `model_dump(exclude_unset=True)` para garantir que apenas os campos explicitamente enviados pelo usuário sejam considerados. Se o usuário não enviar nada válido, retornamos um erro **400 Bad Request**.

```
@router.patch("/{livro_id}", response_model=LivroResposta, responses={404: {"description":
↪  "Livro não encontrado"}, 400:{"description": "Nehum dado válido enviado para
↪  atualização."}})
async def atualizar_parcial(livro_id: UUID, livro_update: LivroPatch, session: SessionDep) ->
↪  LivroResposta:
    if (livro := session.exec(select(Livro).where(Livro.uuid == livro_id)).first()):
        # Filtra para ignorar campos não enviados ou nulos
        update_data = livro_update.model_dump(exclude_unset=True, exclude_none=True)

        if not update_data:
            raise HTTPException(status_code=400, detail="Nenhum dado válido enviado para
            ↪  atualização.")

        for key, value in update_data.items():
            setattr(livro, key, value)

        session.add(livro)
        session.commit()
        session.refresh(livro)
        return LivroResposta.model_validate(livro)

    raise HTTPException(status_code=404, detail="Livro não encontrado.")
```

O Método DELETE: Remoção Permanente

O método **DELETE** é a implementação mais direta: localizamos o livro e solicitamos à sessão que o remova.

```
@router.delete("/{livro_id}", response_model=ConfirmaDelete)
async def deletar_livro(livro_id: UUID, session: SessionDep):
    if (livro := session.exec(select(Livro).where(Livro.uuid == livro_id)).first()):
        titulo_removido = livro.titulo

        session.delete(livro) # Remove o objeto do banco
        session.commit()      # Confirma a exclusão

        return ConfirmaDelete(
            mensagem=f"Livro '{titulo_removido}' deletado com sucesso",
            uuid=livro_id
        )

    raise HTTPException(status_code=404, detail="Livro não encontrado")
```

A Transparência da Refatoração

Um ponto fundamental desta etapa é observar que, embora tenhamos alterado completamente o "back-end" (migrando de dicionários para SQLModel), **o cliente Python não precisou de nenhuma modificação**.

Isso demonstra o poder de uma API bem desenhada: a forma como a API conversa com o mundo exterior permanece o mesmo, permitindo que você troque toda a infraestrutura interna sem quebrar os aplicativos que consomem seus dados. Agora, sua aplicação possui **persistência real**, garantindo que os livros adicionados ou alterados permaneçam salvos no arquivo `database.db` mesmo após o servidor ser reiniciado.

CRUD Completo! Agora sua API é capaz de criar, ler, atualizar (total ou parcialmente) e deletar registros de forma profissional.

15. Paginação e Importação em Lote

Neste capítulo, abordaremos um desafio real no desenvolvimento de APIs: o gerenciamento de grandes volumes de dados. Quando uma base de dados cresce, retornar todos os registros em uma única requisição GET torna o sistema pesado e ineficiente. Para resolver isso, aprenderemos a implementar o esquema de **paginação** e prepararemos nosso ambiente com uma **importação em massa** via arquivo CSV, que está disponibilizado nos materiais complementares.

A Necessidade da Paginação

Atualmente, nosso método GET retorna todos os livros existentes de uma só vez. Se tivéssemos cem mil registros, a requisição seria extremamente lenta. A paginação permite que o cliente escolha quantos itens quer por página (ex: 10 itens), dividindo a carga total em partes menores e gerenciáveis.

Preparação e Limpeza do Ambiente

Antes de implementarmos a paginação, precisamos de uma base de dados volumosa e "limpa". Siga estes passos para resetar o projeto:

1. **Remover dados padrão:** No arquivo `database.py`, remova a função que adicionava livros automaticamente e o dicionário `livros_default_data`.
2. **Ajustar o Ciclo de Vida:** No arquivo `main.py`, remova a chamada de `adicionar_livros_default` dentro do `lifespan`.
3. **Remover Arquivos Antigos:** Delete a implementação anterior (`api.py`) e o arquivo da base de dados (`database.db`) para que o sistema recrie tudo do zero.

Importação de Livros via CSV

Utilizaremos um arquivo chamado `livros.csv` contendo 50 registros para exemplificar a paginação. Como nossa API ainda não verifica se um livro já existe, cada importação adicionará novos registros com UUIDs únicos.

Implementação da Função no `cliente.py` Precisamos criar uma função no cliente que leia o arquivo CSV linha por linha e envie cada registro para o endpoint POST da nossa API.

```
import csv
import requests
```

```
def importar_csv():
    # Solicita o caminho e remove espaços extras
    caminho = input("Caminho do arquivo CSV: ").strip()

    try:
        with open(caminho, encoding="utf-8") as f:
            # DictReader facilita o acesso aos campos pelo nome da coluna
            leitor = csv.DictReader(f, delimiter=";")
            print("Importando livros do CSV...")
            total = 0

            for linha in leitor:
                # Monta o payload para o método POST
                payload = {
                    "autor": linha["autor"],
                    "titulo": linha["titulo"],
                    "editora": linha["editora"],
                    "ano": int(linha["ano"])
                }

                # Envia a requisição para a API
                resp = requests.post(f"{API_URL}/livros", json=payload)

                if resp.status_code == 200 or resp.status_code == 201:
                    total += 1
                else:
                    print(f"Erro ao importar: {linha['titulo']}")
                    tratar_resp(resp)

            print(f"Total de {total} livros importados com sucesso!")

    except FileNotFoundError:
        print("Arquivo não encontrado.")
    except Exception as e:
        print(f"Ocorreu um erro: {e}")
```

O uso do `DictReader` com delimitador `;` é essencial aqui, pois o arquivo CSV fornecido utiliza esse padrão para separar as colunas de autor, título, editora e ano. ## Atualizando o Menu do Cliente

Para acessar a nova funcionalidade, adicione a opção 7 ao menu interativo no script `cliente.py`:

```
# No menu()
print("7. Importar livros do CSV")

# No loop de opções
elif opcao == "7":
    importar_csv()
```

Executando a Importação em Lote

Para realizar o processo, garanta que o arquivo `livros.csv` esteja na pasta raiz do projeto.

1. Inicie a API: `fastapi dev api/main.py`.
2. Execute o cliente: `python cliente/cliente.py`.
3. Escolha a **opção 7** e digite `livros.csv`.

Após a conclusão, você poderá verificar através do navegador (`/docs`) que o GET agora retorna uma lista extensa com todos os 50 livros importados. No próximo capítulo, modificaremos a API para que ela não entregue todos esses dados de uma vez, mas sim de forma paginada.

16. Implementando a Paginação no FastAPI

Neste capítulo, aprenderemos a implementar o método **GET com paginação**, uma técnica essencial para quando a API possui um volume muito grande de dados. O objetivo é permitir que o cliente acesse uma URL como `exemplo.com/livros?page=1` para obter partes controladas dos dados, recebendo metadados sobre o total de itens e páginas diretamente nos cabeçalhos da resposta.

Modificando o Endpoint de Listagem

Para começar a paginação, precisamos ajustar a função `listar_livros` no arquivo `livros_router.py`. Utilizaremos o parâmetro **response** para manipular os cabeçalhos e o parâmetro **page** para capturar a página desejada pelo usuário.

O Uso do `Query` para Validação O FastAPI permite validar parâmetros de consulta diretamente na assinatura da função usando `Query`. Definiremos que o valor padrão da página é **1** e que ele deve ser sempre **maior ou igual a 1** (`ge=1`), o que garante que o framework faça a validação automaticamente sem precisarmos de códigos extras.

```
from fastapi import APIRouter, Query, Response
from sqlmodel import select, func
from api.models import Livro, LivroResposta
from api.database import SessionDep

router = APIRouter(prefix="/livros", tags=["livros"])

@router.get("/", response_model=list[LivroResposta])
async def listar_livros(
    session: SessionDep,
    response: Response,
    page: int = Query(1, ge=1)
) -> list[LivroResposta]:
    # Lógica de paginação virá aqui
    ...
```

Lógica de Cálculo e Consultas ao Banco

A paginação exige alguns cálculos matemáticos para determinar quantos itens pular e onde parar. Definiremos um tamanho fixo de **10 itens por página**.

1. Contando o Total de Itens

Antes de retornar os livros, precisamos saber quantos registros existem no total. Usamos a função `func.count()` do SQLModel para realizar essa contagem de forma eficiente no banco de dados.

2. Calculando o Total de Páginas

Para calcular o total de páginas arredondando para cima, usamos um "truque" matemático: `(total_itens + PAGE_SIZE - 1) // PAGE_SIZE`. Isso garante que, se houver 85 itens, o sistema crie 9 páginas (8 de 10 itens e uma última com 5).

3. Definindo o Offset

O **offset** indica quantos itens o banco de dados deve "pular" antes de começar a ler. O cálculo é `(page - 1) * page_size`. Assim, na página 1 o offset é 0; na página 2, pulamos os 10 primeiros registros.

Implementação Completa do Código

Veja como fica a implementação final integrando a consulta paginada e os cabeçalhos de resposta:

```
@router.get("/", response_model=list[LivroResposta])
async def listar_livros(session: SessionDep, response: Response, page: int = Query(1, ge=1)):
    PAGE_SIZE = 10

    # Busca o total de registros para os metadados
    total_itens = session.exec(select(func.count()).select_from(Livro)).one()
    total_pages = (total_itens + PAGE_SIZE - 1) // PAGE_SIZE

    # Se a página pedida for maior que o total, entrega a última página disponível
    if page > total_pages and total_pages > 0:
        page = total_pages

    # Calcula quantos itens pular
    offset = (page - 1) * PAGE_SIZE

    # Define a query de paginação
    query = select(Livro).limit(PAGE_SIZE).offset(offset)

    # Executa a query limitada e com o pulo (offset)
    livros = session.exec(query).all()

    # Adiciona metadados nos cabeçalhos (headers) da resposta
    response.headers["X-Total-Pages"] = str(total_pages)
    response.headers["X-Total-Items"] = str(total_itens)

    return [LivroResposta.model_validate(l) for l in livros]
```

Testando na Documentação (Swagger)

Ao acessar o `/docs`, você verá que o método GET agora possui o campo **page**.

• **Execução:** Ao executar com `page=1`, a API retorna os primeiros 10 livros e exibe no campo **Response Headers** o `x-total-pages` e o `x-total-items`.

• **Segurança:** Se você tentar passar uma página inválida como `0` ou `-1`, o FastAPI retornará um erro informando que o valor deve ser **maior ou igual a 1**.

• **Limitação:** Caso solicite uma página inexistente (ex: página 10 em uma base de 5 páginas), o sistema está configurado para retornar a última página disponível.

Com essa implementação, sua API está pronta para lidar com grandes bases de dados sem comprometer a performance do servidor ou a experiência do usuário. No próximo capítulo, adaptaremos o cliente para consumir essas páginas sequencialmente.

17. Paginação no Cliente Python

Neste capítulo, adaptaremos o nosso **cliente Python** para consumir a API de forma eficiente, agora que ela implementa a paginação. O objetivo é fazer com que o script `cliente.py` não receba apenas a primeira página, mas seja capaz de identificar quantas páginas existem e percorrê-las automaticamente até obter todos os registros disponíveis.

Lógica de Acúmulo de Dados

Anteriormente, o método `listar_livros` fazia apenas uma requisição simples. Agora, precisamos de uma estrutura que acumule os resultados de múltiplas chamadas em uma única lista para processamento.

Inicialização Definimos a página inicial como 1 e criamos uma lista vazia para armazenar todos os livros que serão recebidos.

```
def listar_livros():
    print("\nListando livros com paginação...")
    pagina_atual = 1
    livros = []
```

Obtendo a Primeira Página e Metadados

A primeira requisição é crucial porque é nela que o servidor nos informa, através dos **headers (cabeçalhos)**, o total de páginas disponíveis.

A Primeira Requisição Fazemos o pedido para a página 1 e verificamos o status da resposta. Se for bem-sucedida, extraímos o valor de `x-total-pages`.

```
    # Primeira requisição (Página 1)
    resp = requests.get(f"{API_URL}/livros?page={pagina_atual}")
    print(f"Página 1 obtida.")
    tratar_resposta(resp)

    if resp.status_code != 200:
        print("Erro ao obter a primeira página.")
        return

    # Usamos extend para adicionar os itens individualmente à lista principal
    dados = resp.json()
    livros.extend(dados)

    # Extraindo o total de páginas do cabeçalho
```

```
# Caso o header não exista, assumimos que há pelo menos 1 página
total_pages = int(resp.headers.get("X-Total-Pages", 1))
```

Percorrendo as Demais Páginas

Com a informação de quantas páginas existem no total, podemos criar um laço de repetição (`for`) que começa na página 2 e vai até o limite informado pelo servidor.

O Loop de Paginação Para cada nova página, repetimos o processo de requisição e anexamos os dados à nossa lista acumuladora.

```
# Loop da página 2 até o total_paginas
for pagina in range(2, total_pages + 1):
    resp = requests.get(f"{API_URL}/livros?page={pagina}")
    print(f"Página {pagina} obtida.")

    tratar_resposta(resp)

    if resp_p.status_code != 200:
        continue

    dados = resp.json()
    livros.extend(dados)
```

Exibição do Resultado Final

Após percorrer todas as páginas, utilizamos a função `len()` na lista `livros_acumulados` para confirmar quantos registros foram recebidos no total.

```
# Exibição consolidada (Fora do FOR anterior)
print(f"\nTotal de livros recebidos: {len(livros)}")
```

Por que usar `extend` em vez de append? Devemos usar o método **extend** para evitar a criação de uma "lista de listas". O `extend` pega os itens de cada página e os coloca diretamente na lista principal, mantendo uma estrutura única e plana, facilitando a contagem e a iteração posterior.

Conclusão do projeto

Ao finalizar esta implementação, você terá construído um sistema completo:

1. **Back-end:** Uma API robusta com **FastAPI**, persistência em **SQLite** via **SQLModel**, validação por **Pydantic** e **Paginação** eficiente.

2. **Front-end (Cliente):** Um script capaz de interagir com todos os métodos HTTP (GET, POST, PUT, PATCH, DELETE) e gerenciar grandes volumes de dados de forma automática.

Este conhecimento forma a base essencial para trabalhar com microsserviços e integrações profissionais no ecossistema Python.

18. Síntese e Consolidação do Aprendizado

Este capítulo final consolida a jornada de construção de uma API robusta utilizando **FastAPI**, desde os fundamentos da comunicação HTTP até a implementação de um sistema persistente e escalável. Ao longo deste curso, você aprendeu a criar uma API "do jeito certo", abandonando códigos mágicos em favor de uma compreensão profunda da lógica de back-end.

1. A Arquitetura: FastAPI, Uvicorn e ASGI

Um dos pilares deste aprendizado é a distinção entre a aplicação e o servidor. O **FastAPI**, como uma aplicação **ASGI** (*Asynchronous Server Gateway Interface*), não entende o protocolo HTTP sozinho; ele depende do **Uvicorn**, o servidor que efetivamente conversa com o mundo exterior (navegadores, scripts ou aplicativos). Essa comunicação assíncrona permite que o sistema processe múltiplas chamadas simultâneas sem bloqueios, garantindo alta performance.

2. Do In-Memory à Persistência com SQLModel

Iniciamos com uma base de dados em memória (dicionários) para focar na lógica da API, mas evoluímos para a persistência real com **SQLModel** e **SQLite**. Essa transição introduziu o conceito de **ORM** (*Object-Relational Mapping*), que traduz objetos Python para o mundo relacional do SQL. Além disso, adotamos o **UUID** como identificador universal, garantindo que os dados permaneçam consistentes e independentes da estrutura física das tabelas, o que é vital para sistemas distribuídos e migrações de banco de dados.

3. O Ciclo de Vida CRUD e a Validação com Pydantic

O domínio dos métodos HTTP foi o coração do desenvolvimento. Através do **Pydantic**, implementamos validações automáticas que garantem a segurança e a integridade dos dados enviados. Reforçamos a semântica de cada operação:

• **GET:** Extração de dados (todos ou específicos via ID/UUID).

• **POST:** Criação de novos registros, onde a responsabilidade de gerar IDs e UUIDs recai sobre o servidor.

• **PUT:** Substituição integral de um recurso.

• **PATCH:** Atualização parcial através da mesclagem de dados.

• **DELETE:** Remoção permanente de registros.

4. Organização Modular e Escalabilidade

Para tornar o projeto profissional, reestruturamos o código em pastas separadas (`api`, `cliente`, `routers`) e utilizamos o **APIRouter** para organizar as rotas. Implementamos a **Injeção de Dependência** para gerenciar sessões de banco de dados e o **Lifespan** para tarefas automatizadas no início da aplicação, como a criação automática de tabelas.

5. Lidando com Grandes Volumes: Paginação

Finalmente, enfrentamos o desafio da performance com a **Paginação**. Em vez de retornar milhares de registros de uma vez, o servidor agora entrega dados em blocos controlados (páginas), utilizando cálculos de `limit` e `offset` no banco de dados. O uso de **headers customizados** (`x-total-pages`, `x-total-items`) permite que o cliente saiba exatamente como navegar pelos dados de forma eficiente.

Conclusão

Ao construir simultaneamente a API e o **Cliente Python**, você obteve a visão completa do ciclo de vida de um serviço web. Este conhecimento não é apenas teórico; é uma base prática e segura que fornece a segurança necessária para você trabalhar com back-end e sistemas reais. Agora, você entende não apenas como o código funciona, mas o que acontece por trás de cada requisição no coração do back-end. A partir daqui, você está pronto para explorar tópicos mais avançados, como autenticação, chaves de API e implantação em produção.